CSC 370 PaperTrade, Minimal Basis and FD Projection

Dryden Bryson

November 4, 2026

Overview

This document does three things for the FDs of the 8-table `stock_trading` database:

1. derives a **minimal basis** G for the full FD set F;
2. **projects** F onto every relation used in the *Lossless Join Proof*: the four universal relations U_T, U_O, U_H and U_W, the intermediate relations of their BCNF decompositions, and the eight final relations `user`, `portfolio`, `exchange`, `stock`, `stock_price_history`, `watchlist`, `transaction` and `limit_order`. Each projection is reduced to a minimal basis;
3. shows that the eight final relations together **preserve** every FD of F.

The projections show that every BCNF violation used in the decomposition really holds in the relation where it was used.

Notation

As in the *Lossless Join Proof*, A is the set of every attribute in the database. Three attribute names are used by two tables, so in A they are renamed by table:

Column	Tables	Renamed to
`quantity`	`transaction`, `limit_order`	t_quantity, o_quantity
`target_price`	`watchlist`, `limit_order`	w_target_price, o_target_price
`created_at`	`portfolio`, `limit_order`	p_created_at, o_created_at

Every other shared name (`user_id`, `portfolio_id`, `ticker`, `exchange_code`) is a foreign key and means the same thing in every table, so it is one attribute of A. We group attributes that always appear together:

$$N = \{\text{name}, \text{email}, \text{cash_balance}\}$$
$$E = \{\text{exchange_name}, \text{country}, \text{currency}\}$$
$$W = \{\text{w_target_price}, \text{added_at}\}$$
$$H = \{\text{opening_price}, \text{closing_price}, \text{high_price}, \text{low_price}, \text{volume}\}$$
$$O = \{\text{order_type}, \text{o_target_price}, \text{o_quantity}, \text{status}, \text{o_created_at}\}$$
$$Q = \{\text{portfolio_name}, \text{p_created_at}\}$$
$$S = \{\text{company_name}, \text{sector}\}$$
$$T = \{\text{transaction_type}, \text{t_quantity}, \text{price}, \text{executed_at}\}$$

$$A = \{\text{user_id}, \text{portfolio_id}, \text{exchange_code}, \text{ticker}, \text{price_date}, \text{transaction_id}, \text{order_id}\} \cup N \cup Q \cup E \cup S \cup H \cup W \cup T \cup O$$

That is 33 attributes: the 42 columns of `sql/ddl.sql`, minus the 9 repeats of the four foreign keys.

The FD set F, from the *Functional Dependencies and Candidate Keys* document:

- FD_1: user_id → name, email, cash_balance
- FD_2: email → user_id
- FD_3: portfolio_id → user_id, portfolio_name, p_created_at
- FD_4: user_id, portfolio_name → portfolio_id
- FD_5: exchange_code → exchange_name, country, currency
- FD_6: ticker → company_name, sector, exchange_code
- FD_7: ticker, price_date → opening_price, closing_price, high_price, low_price, volume
- FD_8: user_id, ticker → w_target_price, added_at
- FD_9: transaction_id → portfolio_id, ticker, transaction_type, t_quantity, price, executed_at
- FD_{10}: order_id → portfolio_id, ticker, order_type, o_target_price, o_quantity, status, o_created_at

1 Minimal Basis of F

A set of FDs G is a **minimal basis** for F if G is equivalent to F ($G^+ = F^+$) and:

(1) every right-hand side is a single attribute;

(2) no FD can be removed, because no $X \to B$ in G follows from the other FDs of G;

(3) no left-hand side can be reduced, because for no $X \to B$ in G and $C \in X$ does $X \setminus \{C\} \to B$ follow from G.

We build G in three steps, one for each condition.

Step 1: Single-Attribute Right-Hand Sides

By decomposition, each FD $X \to B_1 \cdots B_k$ is equivalent to the k FDs $X \to B_1, \ldots, X \to B_k$ (and union recombines them). Splitting FD_1 to FD_{10} gives 34 FDs:

From	Label	FD
FD_1	g_1–g_3	user_id → B for each $B \in N$
FD_2	g_4	email → user_id
FD_3	g_5, g_6, g_7	portfolio_id → user_id, portfolio_id → portfolio_name, portfolio_id → p_created_at
FD_4	g_8	user_id, portfolio_name → portfolio_id
FD_5	g_9–g_{11}	exchange_code → B for each $B \in E$
FD_6	g_{12}, g_{13}, g_{14}	ticker → company_name, ticker → sector, ticker → exchange_code
FD_7	g_{15}–g_{19}	ticker, price_date → B for each $B \in H$
FD_8	g_{20}, g_{21}	user_id, ticker → B for each $B \in W$
FD_9	g_{22}–g_{27}	transaction_id → B for each $B \in \{\text{portfolio_id}, \text{ticker}\} \cup T$
FD_{10}	g_{28}–g_{34}	order_id → B for each $B \in \{\text{portfolio_id}, \text{ticker}\} \cup O$

In the last two rows the first two labels are the foreign keys: g_{22} is transaction_id → portfolio_id, g_{23} is transaction_id → ticker, and likewise g_{28} and g_{29} for order_id.

Step 2: Reduce Left-Hand Sides

Only g_8, g_{15}–g_{19} and g_{20}, g_{21} have more than one attribute on the left. For each, we test whether either attribute alone already determines the right-hand side, using closures under the full set G:

$$\begin{aligned}
\{\text{user_id}\}^+ &= \{\text{user_id}\} \cup N && (g_1\text{–}g_3, \text{ then } g_4 \text{ adds nothing new})\\
\{\text{portfolio_name}\}^+ &= \{\text{portfolio_name}\} && (\text{no FD has a left-hand side inside } \{\text{portfolio_name}\})\\
\{\text{ticker}\}^+ &= \{\text{ticker}, \text{exchange_code}\} \cup S \cup E && (g_{12}\text{–}g_{14}, \text{ then } g_9\text{–}g_{11})\\
\{\text{price_date}\}^+ &= \{\text{price_date}\} && (\text{no FD has a left-hand side inside } \{\text{price_date}\})
\end{aligned}$$

- g_8: neither $\{\text{user_id}\}^+$ nor $\{\text{portfolio_name}\}^+$ contains portfolio_id. A user can have many portfolios, and two users can use the same portfolio name (C.2.4).
- g_{15}–g_{19}: neither $\{\text{ticker}\}^+$ nor $\{\text{price_date}\}^+$ meets H. A price record belongs to one stock on one day.
- g_{20}, g_{21}: neither $\{\text{user_id}\}^+$ nor $\{\text{ticker}\}^+$ meets W. A watchlist entry belongs to one user and one stock together.

No left-hand side can be reduced.

Step 3: Remove Redundant FDs

An FD $X \to B$ is redundant if $B \in X^+$, where the closure is computed with $X \to B$ itself removed.

Shortcut. If $B \notin X$ and $X \to B$ is the *only* FD with B on its right-hand side, it cannot be redundant. Without it, the closure algorithm has no FD that adds B, and B is not in the starting set X.

In G, only three attributes are on the right-hand side of more than one FD:

- user_id: g_4 and g_5;
- portfolio_id: g_8, g_{22} and g_{28};
- ticker: g_{23} and g_{29}.

By the shortcut, the other 27 FDs are not redundant. Note also that transaction_id and order_id are on no right-hand side, so a closure only contains them if it starts with them. We test the remaining seven directly:

- g_4 (email → user_id): without g_4, no FD has a left-hand side inside $\{\text{email}\}$, so $\{\text{email}\}^+ = \{\text{email}\}$. Not redundant.
- g_5 (portfolio_id → user_id): without g_5, $\{\text{portfolio_id}\}^+ = \{\text{portfolio_id}\} \cup Q$ by g_6, g_7. Reaching user_id would take g_4, which needs email, which only g_1–g_3 give, which need user_id. Not redundant.
- g_8 (user_id, portfolio_name → portfolio_id): without g_8, $\{\text{user_id}, \text{portfolio_name}\}^+ = \{\text{user_id}, \text{portfolio_name}\} \cup N$. The other FDs for portfolio_id need transaction_id or order_id, which are never reached. Not redundant.
- g_{22} (transaction_id → portfolio_id): without g_{22},

  $$\{\text{transaction_id}\}^+ = \{\text{transaction_id}, \text{ticker}, \text{exchange_code}\} \cup T \cup S \cup E \quad (g_{23} \text{ to } g_{27}, \text{ then } g_{12}\text{–}g_{14}, g_9\text{–}g_{11})$$

  g_8 needs user_id, which needs portfolio_id (g_5) or email (g_4, which needs user_id), and g_{28} needs order_id. Not redundant.
- g_{23} (transaction_id → ticker): without g_{23}, $\{\text{transaction_id}\}^+ = \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}\} \cup T \cup Q \cup N$. The only other FD for ticker is g_{29}, which needs order_id. Not redundant.
- g_{28}, g_{29}: the same argument with order_id and O in place of transaction_id and T. Not redundant.

Result

No FD can be removed and no left-hand side can be reduced, so all 34 FDs of Step 1 form a minimal basis $G = \{g_1, \ldots, g_{34}\}$ of F.

Minimal bases are not unique. g_1 (user_id → name) could be replaced by email → name, and the set would still be minimal and equivalent to F, since user_id → email → name recovers g_1 by transitivity. We keep g_1 because it matches how `user` is keyed.

2 Projecting FDs

The Projection Algorithm

The projection of F onto a relation $R \subseteq A$ is the set of all FDs $X \to B$ in F^+ with $X \cup \{B\} \subseteq R$. It is computed by taking every $X \subseteq R$, finding the closure X^+ under F in all of A, and adding

$$X \to (X^+ \cap R) \setminus X$$

The closure must be taken in A, not in R, because a derivation can pass through attributes outside R. The result is then reduced to a minimal basis.

Trying all $2^{|R|}$ subsets is unnecessary. These rules let us skip subsets without missing any FD:

(R1) **Empty set:** no FD has an empty left-hand side, so $\emptyset^+ = \emptyset$ gives nothing.

(R2) **Supersets of a superkey:** if $X^+ \supseteq R$, every $Y \supseteq X$ only gives FDs that follow from $X \to R$ by augmentation and decomposition.

(R3) **Attributes on no left-hand side:** let

$$L = \{\text{user_id}, \text{email}, \text{portfolio_id}, \text{portfolio_name}, \text{exchange_code}, \text{ticker}, \text{price_date}, \text{transaction_id}, \text{order_id}\},$$

the attributes on some left-hand side of G. If $X = Y \cup Z$ with $Z \cap L = \emptyset$, adding Z cannot fire any new FD, so $X^+ = Y^+ \cup Z$. Only subsets of $R \cap L$ need to be tried.

(R4) **Single attributes combine by union:** if X^+ equals the union of $\{C\}^+$ over $C \in X$, the FD from X follows from the single-attribute FDs by augmentation and union. The union of single-attribute closures is already closed under every FD with a one-attribute left-hand side, so X^+ can only be bigger if it contains one of the three two-attribute left-hand sides $\{\text{user_id}, \text{portfolio_name}\}$, $\{\text{ticker}, \text{price_date}\}$ or $\{\text{user_id}, \text{ticker}\}$ without any single member of X reaching it. So beyond single attributes we only test $\{\text{user_id}, \text{portfolio_name}\}$, $\{\text{ticker}, \text{price_date}\}$, $\{\text{user_id}, \text{ticker}\}$ and $\{\text{portfolio_id}, \text{ticker}\}$ (portfolio_id also reaches user_id), when they lie in R.

(R5) **email and user_id:** each determines the other, and every relation that contains email also contains user_id. A set using email has the same closure as the set using user_id instead, so we test the user_id version only, plus $\{\text{email}\}$ itself.

Closures Used Below

These are computed once, under G in all of A:

$$
\begin{aligned}
\{\text{user_id}\}^+ = \{\text{email}\}^+ &= \{\text{user_id}\} \cup N && (g_1\text{–}g_3;\ g_4 \text{ then } g_1\text{–}g_3)\\
\{\text{portfolio_id}\}^+ &= \{\text{portfolio_id}, \text{user_id}\} \cup Q \cup N && (g_5\text{–}g_7,\ \text{then } g_1\text{–}g_3)\\
\{\text{portfolio_name}\}^+ &= \{\text{portfolio_name}\}\\
\{\text{exchange_code}\}^+ &= \{\text{exchange_code}\} \cup E && (g_9\text{–}g_{11})\\
\{\text{ticker}\}^+ &= \{\text{ticker}, \text{exchange_code}\} \cup S \cup E && (g_{12}\text{–}g_{14},\ \text{then } g_9\text{–}g_{11})\\
\{\text{price_date}\}^+ &= \{\text{price_date}\}\\
\{\text{user_id}, \text{portfolio_name}\}^+ &= \{\text{portfolio_id}\}^+ && (g_8,\ \text{then as above})\\
\{\text{ticker}, \text{price_date}\}^+ &= \{\text{ticker}\}^+ \cup \{\text{price_date}\} \cup H && (g_{15}\text{–}g_{19})\\
\{\text{user_id}, \text{ticker}\}^+ &= \{\text{user_id}\}^+ \cup \{\text{ticker}\}^+ \cup W && (g_{20}, g_{21})\\
\{\text{portfolio_id}, \text{ticker}\}^+ &= \{\text{portfolio_id}\}^+ \cup \{\text{ticker}\}^+ \cup W && (g_5,\ \text{then } g_{20}, g_{21})\\
\{\text{transaction_id}\}^+ &= \{\text{transaction_id}\} \cup T \cup \{\text{portfolio_id}, \text{ticker}\}^+ && (g_{22}\text{–}g_{27},\ \text{then as above})\\
\{\text{order_id}\}^+ &= \{\text{order_id}\} \cup O \cup \{\text{portfolio_id}, \text{ticker}\}^+ && (g_{28}\text{–}g_{34},\ \text{then as above})
\end{aligned}
$$

Notation. For a relation R, write $G[R] = \{g \in G : \text{every attribute of } g \text{ is in } R\}$. Every projection below turns out to have $G[R]$ as its minimal basis: no FD outside G survives the reduction. Where a derived FD could have appeared, we point it out.

3 Projections onto the Universal Relations

The *Lossless Join Proof* uses one universal relation per kind of fact:

$$
\begin{aligned}
U_T &= \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}, \text{exchange_code}\} \cup N \cup Q \cup S \cup E \cup T && \text{(one row per trade)}\\
U_O &= \{\text{order_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}, \text{exchange_code}\} \cup N \cup Q \cup S \cup E \cup O && \text{(one row per limit order)}\\
U_H &= \{\text{ticker}, \text{price_date}, \text{exchange_code}\} \cup S \cup E \cup H && \text{(one row per daily price)}\\
U_W &= \{\text{user_id}, \text{ticker}, \text{exchange_code}\} \cup N \cup S \cup E \cup W && \text{(one row per watchlist entry)}
\end{aligned}
$$

3.1 U_T

$$U_T \cap L = \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}, \text{email}, \text{portfolio_name}, \text{ticker}, \text{exchange_code}\}$$

Every set containing transaction_id is skipped by (R2).

X	$(X^+ \cap U_T) \setminus X$	**Projected FD**
$\{\text{transaction_id}\}$	everything else ($W \not\subseteq U_T$)	superkey
$\{\text{portfolio_id}\}$	$\{\text{user_id}\} \cup Q \cup N$	portfolio_id $\to$ user_id, Q, N
$\{\text{user_id}\}$	N	user_id $\to N$
$\{\text{email}\}$	$\{\text{user_id}\} \cup N \setminus \{\text{email}\}$	email $\to$ user_id, name, cash_balance
$\{\text{portfolio_name}\}$	$\emptyset$	none
$\{\text{ticker}\}$	$\{\text{exchange_code}\} \cup S \cup E$	ticker $\to$ exchange_code, S, E
$\{\text{exchange_code}\}$	E	exchange_code $\to E$
$\{\text{user_id}, \text{portfolio_name}\}$	$\{\text{portfolio_id}, \text{p_created_at}\} \cup N$	user_id, portfolio_name $\to$ portfolio_id, p_created_at, N
$\{\text{user_id}, \text{ticker}\}$	union of single rows	nothing new ($W \not\subseteq U_T$)
$\{\text{portfolio_id}, \text{ticker}\}$	union of single rows	nothing new ($W \not\subseteq U_T$)

Minimal basis of the projection. After splitting right-hand sides:

- transaction_id $\to B$ for $B \notin \{\text{portfolio_id}, \text{ticker}\} \cup T$ is redundant: it follows through portfolio_id or ticker.
- portfolio_id $\to B$ for $B \in N$ is redundant: portfolio_id $\to$ user_id $\to B$.
- email $\to$ name and email $\to$ cash_balance are redundant: email $\to$ user_id $\to B$.
- ticker $\to B$ for $B \in E$ is redundant: ticker $\to$ exchange_code $\to B$.
- user_id, portfolio_name $\to B$ for $B \neq$ portfolio_id is redundant: it follows through portfolio_id or reduces to user_id $\to B$.

$$F_{U_T} = \{g_1\text{–}g_{14}\} \cup \{g_{22}\text{–}g_{27}\} = G[U_T]$$

A derived FD that stays out. transaction_id $\to W$ is in F^+: a trade fixes the portfolio, so the user, and the stock, and FD_8 then fixes the watchlist entry. It does not project onto U_T only because $W \not\subseteq U_T$. A user does not have to watch a stock to trade it, so U_T leaves W out.

3.2 U_O

Identical to U_T, with order_id and O in place of transaction_id and T:

$$F_{U_O} = \{g_1\text{–}g_{14}\} \cup \{g_{28}\text{–}g_{34}\} = G[U_O]$$

3.3 U_H

$U_H \cap L = \{\text{ticker}, \text{price_date}, \text{exchange_code}\}$.

X	$(X^+ \cap U_H) \setminus X$	**Projected FD**
{ticker}	{exchange_code} $\cup S \cup E$	ticker $\to$ exchange_code, S, E
{price_date}	$\emptyset$	none
{exchange_code}	E	exchange_code $\to E$
{ticker, price_date}	everything else	superkey

Removing ticker $\to E$ (it follows through exchange_code) and the parts of the superkey FD that follow from ticker alone:

$$F_{U_H} = \{g_9\text{–}g_{19}\} = G[U_H]$$

3.4 U_W

$U_W \cap L = \{\text{user_id}, \text{email}, \text{ticker}, \text{exchange_code}\}$. The single-attribute rows are the same as in U_T (user_id $\to N$, email $\to$ user_id, ticker $\to$ exchange_code, S, E, exchange_code $\to E$), and $\{\text{user_id}, \text{ticker}\}^+ \supseteq U_W$, so {user_id, ticker} is a superkey. Reducing as before:

$$F_{U_W} = \{g_1\text{–}g_4\} \cup \{g_9\text{–}g_{14}\} \cup \{g_{20}, g_{21}\} = G[U_W]$$

4 Projections onto the Intermediate Relations

Each step of the decomposition removes one group of attributes. The tables give $(X^+ \cap R) \setminus X$ for every X the rules leave, and the last row names the BCNF violation that the next step removes: an FD whose left-hand side is not a superkey of R.

4.1 Decomposition of U_T

$T_1 = U_T \setminus N$, $\quad T_2 = T_1 \setminus E$, $\quad T_3 = T_2 \setminus (S \cup \{\text{exchange_code}\}) = \{\text{transaction_id}, \text{portfolio_id}, \text{user_id}, \text{ticker}\} \cup Q \cup T$

X	T_1	T_2	T_3
{transaction_id}	superkey	superkey	superkey
{portfolio_id}	{user_id} $\cup Q$	{user_id} $\cup Q$	{user_id} $\cup Q$
{user_id}	$\emptyset$ (N removed)	$\emptyset$	$\emptyset$
{portfolio_name}	$\emptyset$	$\emptyset$	$\emptyset$
{user_id, portfolio_name}	{portfolio_id, p_created_at}	{portfolio_id, p_created_at}	{portfolio_id, p_created_at}
{ticker}	{exchange_code} $\cup S \cup E$	{exchange_code} $\cup S$	$\emptyset$
{exchange_code}	E	$\emptyset$ (E removed)	not in T_3
{user_id, ticker}	nothing new	nothing new	nothing new
{portfolio_id, ticker}	nothing new	nothing new	nothing new
Minimal basis	g_5–g_{14}, g_{22}–g_{27}	g_5–g_8, g_{12}–g_{14}, g_{22}–g_{27}	g_5–g_8, g_{22}–g_{27}
Next violation	g_9–g_{11}	g_{12}–g_{14}	g_5–g_7

The violations are exchange_code $\to E$ in T_1, ticker $\to$ exchange_code, S in T_2, and portfolio_id $\to$ user_id, Q in T_3. In each case the left-hand side does not reach transaction_id, the only key, so it is not a superkey. g_8 is also a violation in T_3; it ends up inside `portfolio` along with g_5–g_7.

Why exchange goes before stock. T_2 is formed by removing E while exchange_code is still present. Had we split off `stock` first, the remaining relation would hold ticker and E but not exchange_code, and its projection would contain the derived FD ticker $\to E$, which is not in G. That relation would need another split and would not match our tables. Removing groups from the "leaves" of the FD graph inward (user, exchange, stock, portfolio) keeps every projection equal to $G[R]$.

4.2 Decomposition of U_O

Identical to U_T, with $O_1 = U_O \setminus N$, $O_2 = O_1 \setminus E$ and $O_3 = O_2 \setminus (S \cup \{\text{exchange_code}\})$, and g_{28}–g_{34} in place of g_{22}–g_{27}. The violations removed are again g_9–g_{11}, g_{12}–g_{14} and g_5–g_7.

4.3 Decomposition of U_H

$H_1 = U_H \setminus E = \{\text{ticker}, \text{price_date}, \text{exchange_code}\} \cup S \cup H$. In H_1, ticker gives {exchange_code} $\cup S$, exchange_code and price_date give nothing, and {ticker, price_date} is a superkey. Minimal basis g_{12}–g_{19}. Next violation: g_{12}–g_{14}.

4.4 Decomposition of U_W

$W_1 = U_W \setminus N$ and $W_2 = W_1 \setminus E$. In both, user_id gives nothing (as N is removed) and {user_id, ticker} is a superkey. In W_1, ticker gives {exchange_code} $\cup S \cup E$ and exchange_code gives E: minimal basis g_9–g_{14}, g_{20}, g_{21}, next violation g_9–g_{11}. In W_2, ticker gives {exchange_code} $\cup S$: minimal basis g_{12}–g_{14}, g_{20}, g_{21}, next violation g_{12}–g_{14}.

5 Projections onto the Final Relations

Relation	X	$(X^+ \cap R) \setminus X$	Minimal basis
`user`	{user_id}	N	
	{email}	{user_id, name, cash_balance}	g_1–g_4
`portfolio`	{portfolio_id}	{user_id} $\cup\, Q$	
	{user_id}, {portfolio_name}	$\emptyset$	
	{user_id, portfolio_name}	{portfolio_id, p_created_at}	g_5–g_8
`exchange`	{exchange_code}	E	g_9–g_{11}
`stock`	{ticker}	{exchange_code} $\cup\, S$	
	{exchange_code}	$\emptyset$ (E is not in `stock`)	g_{12}–g_{14}
`stock_price_history`	{ticker}, {price_date}	$\emptyset$	
	{ticker, price_date}	H	g_{15}–g_{19}
`watchlist`	{user_id}, {ticker}	$\emptyset$	
	{user_id, ticker}	W	g_{20}, g_{21}
`transaction`	{transaction_id}	{portfolio_id, ticker} $\cup\, T$	
	{portfolio_id}, {ticker}, {portfolio_id, ticker}	$\emptyset$	g_{22}–g_{27}
`limit_order`	{order_id}	{portfolio_id, ticker} $\cup\, O$	
	{portfolio_id}, {ticker}, {portfolio_id, ticker}	$\emptyset$	g_{28}–g_{34}

Notes on the reductions:

- `user`: email → name and email → cash_balance are redundant (email → user_id → B). The other four each have the only FD for their right-hand side.
- `portfolio`: user_id, portfolio_name → p_created_at is redundant (→ portfolio_id → p_created_at). g_8 cannot be reduced, because neither user_id nor portfolio_name alone gives anything here.
- `transaction` and `limit_order`: {portfolio_id, ticker} determines nothing. In A it reaches user_id and then W, but neither is in these relations. This is why one portfolio can trade the same stock, or place orders on it, many times.

Every left-hand side in each final minimal basis is a superkey of its relation, so all eight relations are in BCNF, matching the *Functional Dependencies and Candidate Keys* document.

6 Dependency Preservation

The minimal bases of the eight final projections are disjoint, and together they are exactly G:

$$F_{\texttt{user}} \cup F_{\texttt{portfolio}} \cup F_{\texttt{exchange}} \cup F_{\texttt{stock}} \cup F_{\texttt{stock_price_history}} \cup F_{\texttt{watchlist}} \cup F_{\texttt{transaction}} \cup F_{\texttt{limit_order}} = \{g_1, \ldots, g_{34}\} = G$$

G is a minimal basis of F, so $G^+ = F^+$. Every FD of F can therefore be enforced inside a single relation, by that relation's primary key or a `UNIQUE` constraint, with no join needed. The decomposition is **dependency preserving**, as well as lossless.

Relation	Minimal basis of projection	From	Enforced in `sql/ddl.sql` by
`user`	g_1–g_4	FD_1, FD_2	`pk_user`, `uq_user_email`
`portfolio`	g_5–g_8	FD_3, FD_4	`pk_portfolio`, `uq_portfolio_user_name`
`exchange`	g_9–g_{11}	FD_5	`pk_exchange`
`stock`	g_{12}–g_{14}	FD_6	`pk_stock`
`stock_price_history`	g_{15}–g_{19}	FD_7	`pk_stock_price_history`
`watchlist`	g_{20}, g_{21}	FD_8	`pk_watchlist`
`transaction`	g_{22}–g_{27}	FD_9	`pk_transaction`
`limit_order`	g_{28}–g_{34}	FD_{10}	`pk_limit_order`